CS 522 – Selected Topics in CS

Artificial Neural Network *(ANN)*

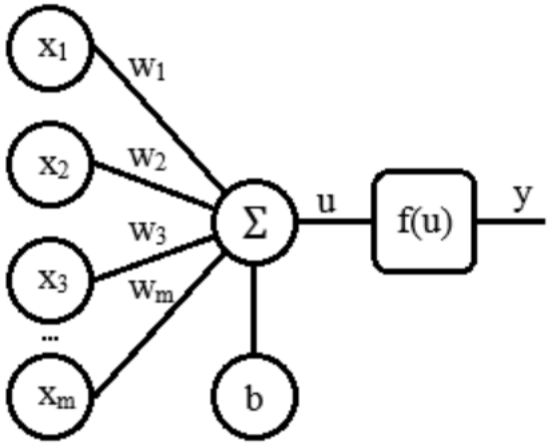
March 20, 2022

Outline

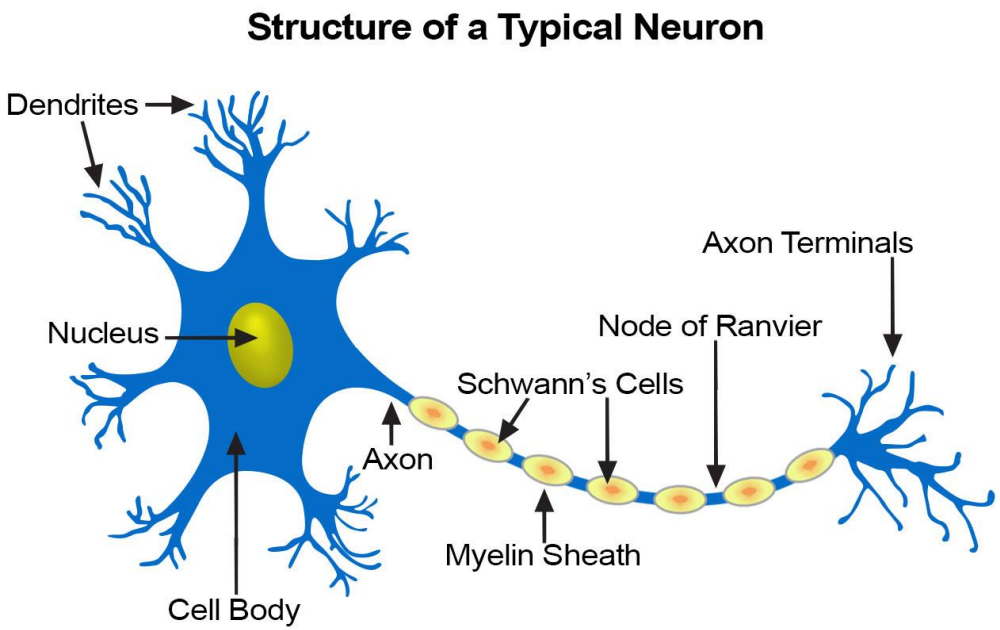
- What is ANN?
- Components of ANN
- How it works?
- Activation Functions
- Types of ANN
  - Feedforward NN
  - Back-Propagation in NN
- Role of Activation Function in Back-Propagation
- Basics for creating Deep NN model

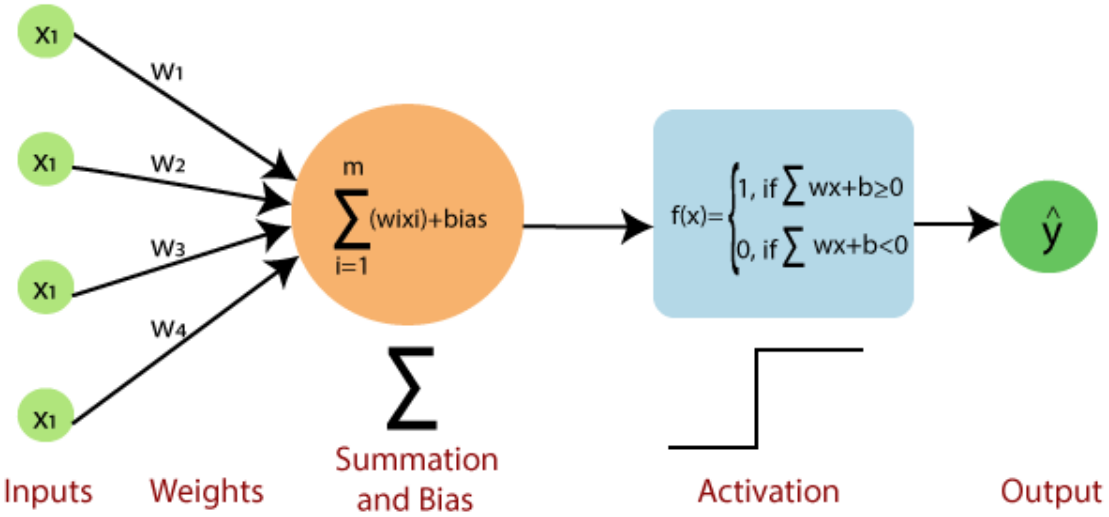
What is Neural Network?

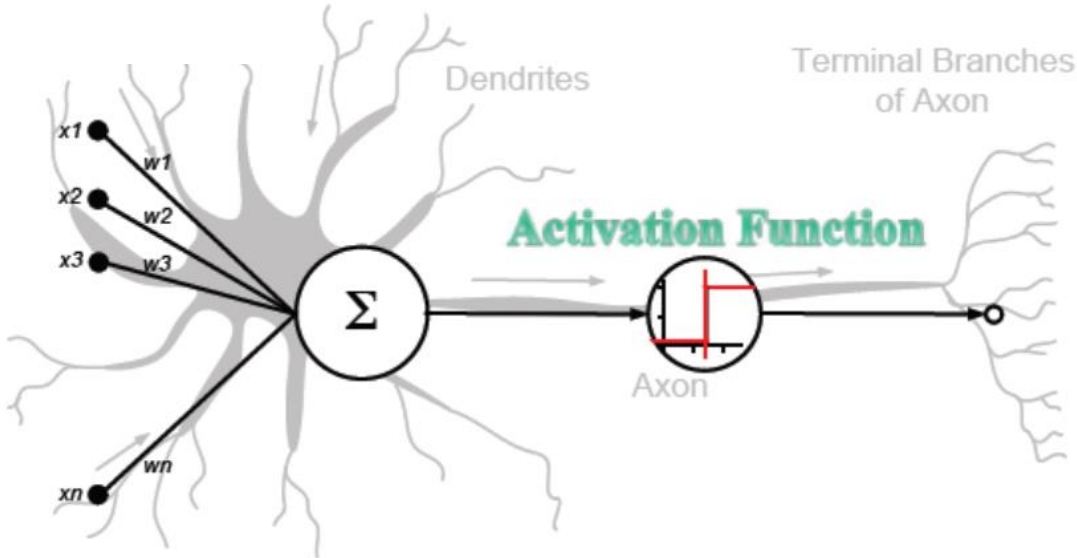
- Neural Networks as a mathematical function that maps a given input to a desired output.
- Neural Networks consist of the following components
  - An ***input layer, x***
  - An arbitrary amount of ***hidden layers***
  - An ***output layer, ŷ***
  - A set of ***weights*** and ***biases*** between each layer, ***W and b***
  - A choice of ***activation function*** for each hidden layer, ***σ***.



What is Neuron?





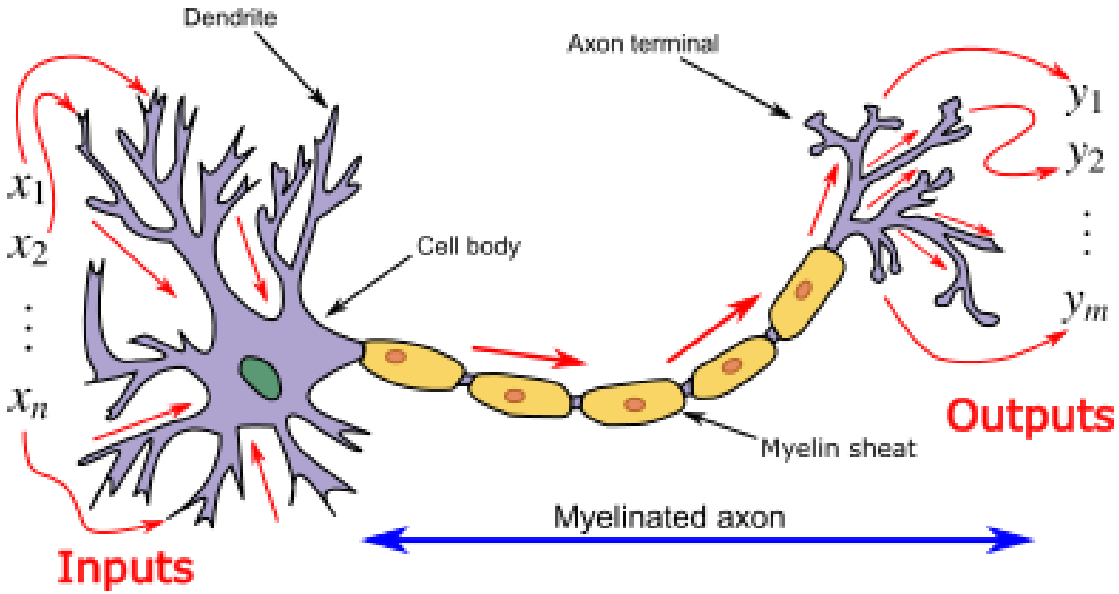


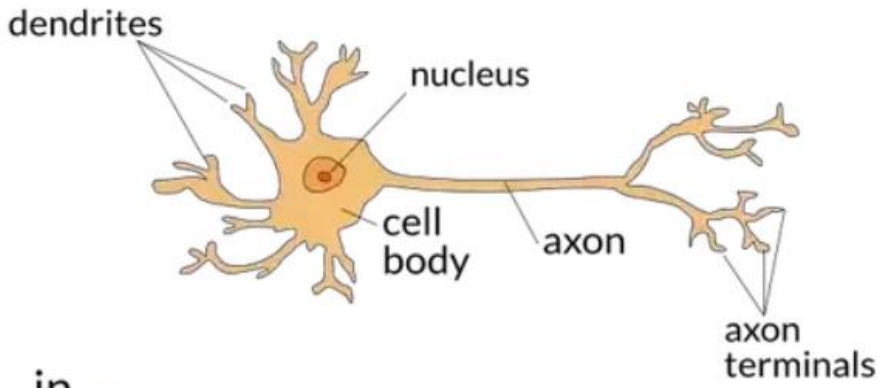
From Biological to Artificial Neurons

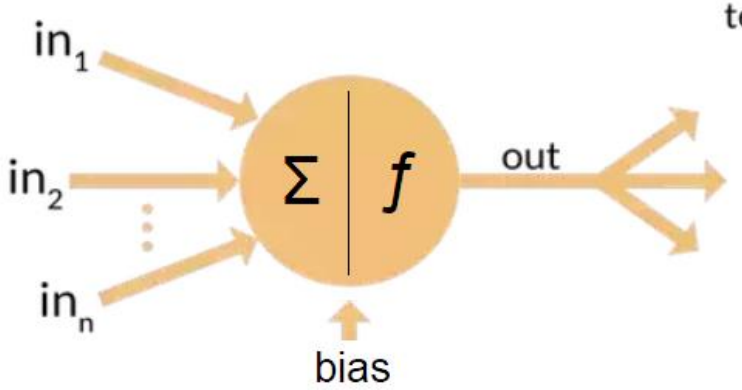
Behavior of an artificial neural network to any particular input depends upon:

- structure of each ***node*** (***activation function***)
- structure of the ***network*** (***architecture***)
- ***weights*** on each of the ***connections***

.... these must be learned !







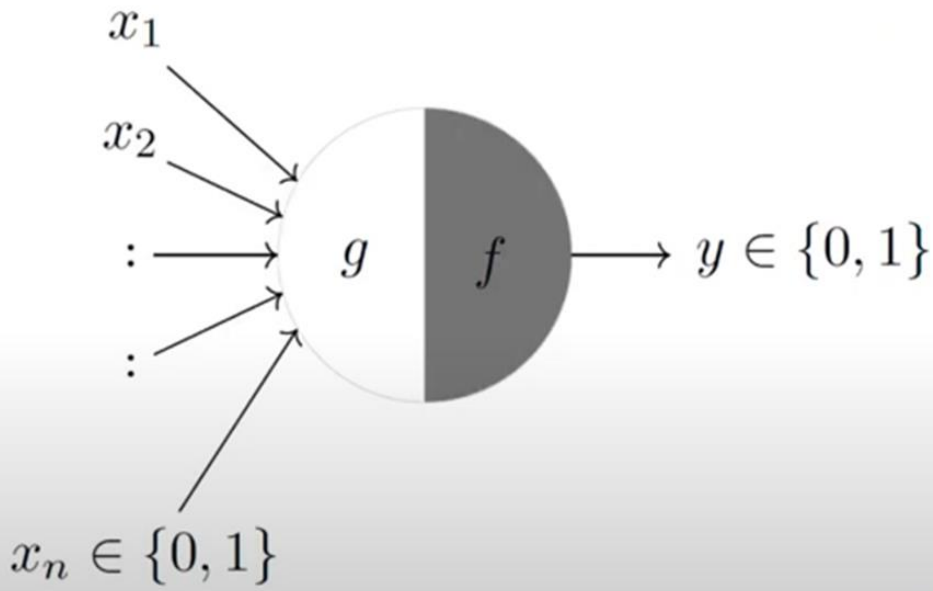
McCulloch-Pitts Neuron (1st ver.)

- McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)
- g aggregates the inputs and the function f takes a decision based on this aggregation
- The inputs can be excitatory or inhibitory
- $y = 0$ if any x_i is inhibitory, else

$$g(x_1, x_2, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

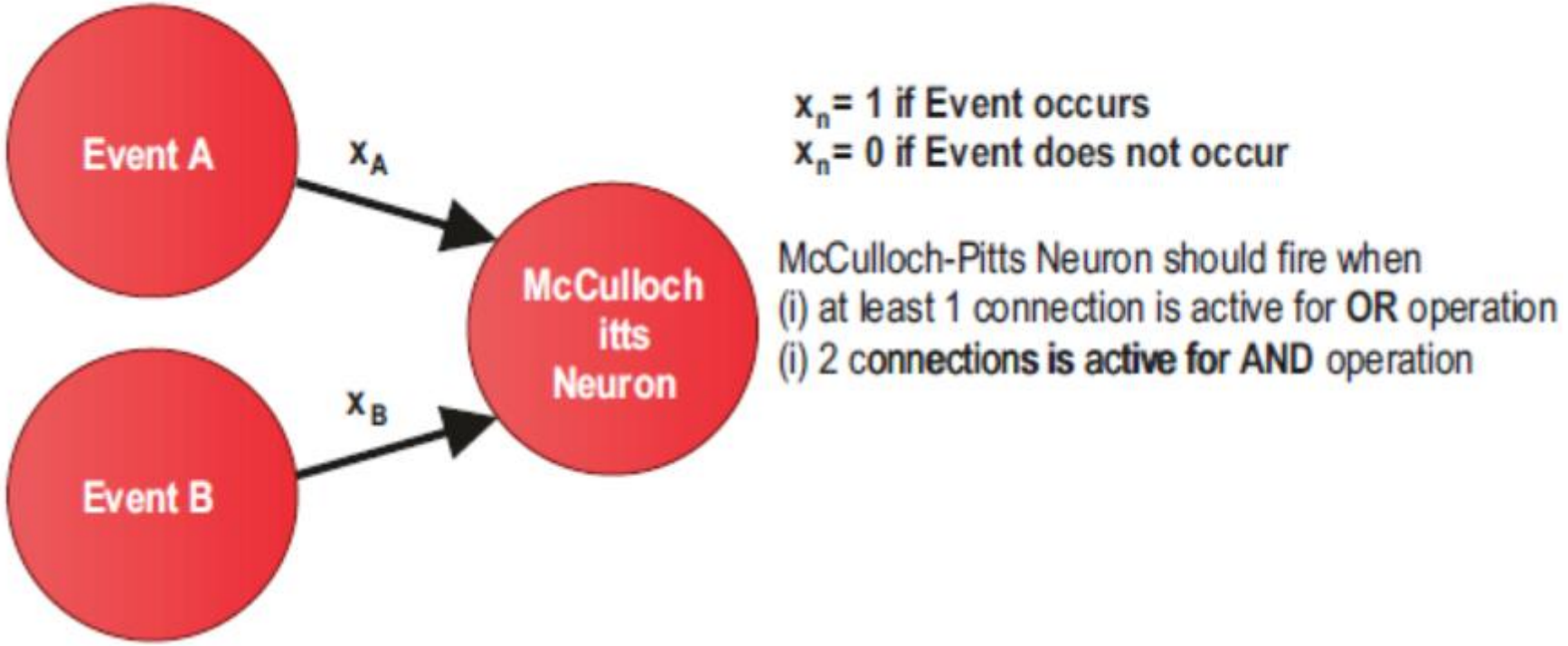
$$y = f(g(\mathbf{x})) = 1 \; if \; g(\mathbf{x}) \geq \theta$$
$$= 0 \; if \; g(\mathbf{x}) < \theta$$

- θ is a thresholding parameter



McCulloch-Pitts Neuron (1st ver.)

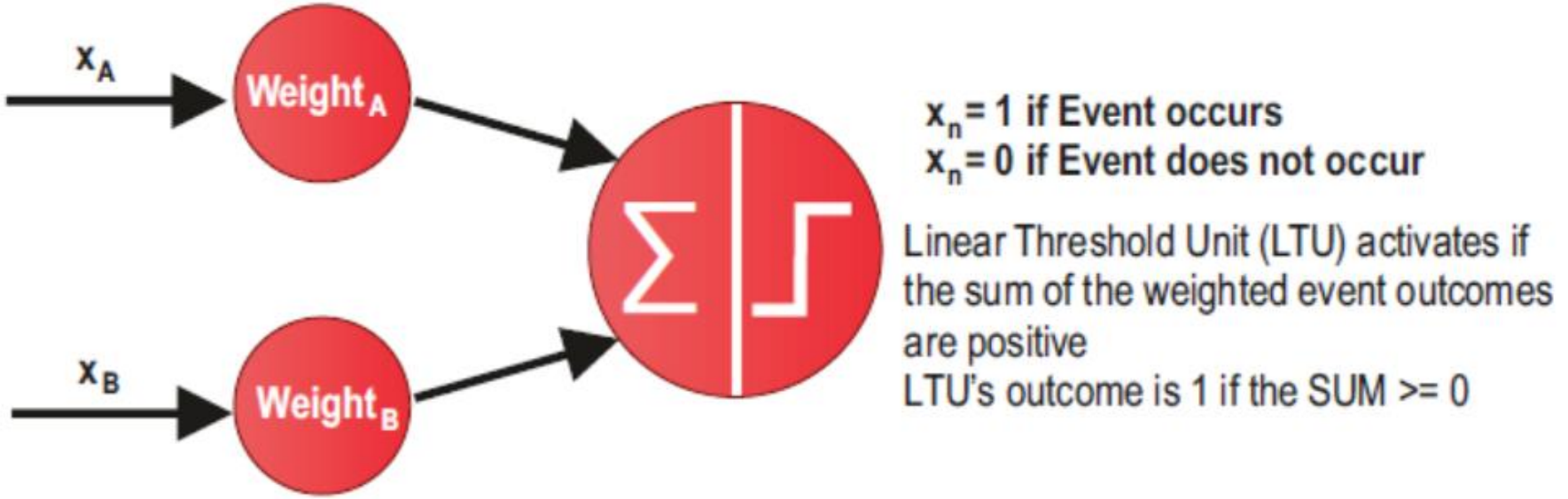
- A visual example for OR and AND operations with McCulloch Pitts Neuron is shown in Figure



Since the inputs from the events in McCulloch Pitts Neuron can only be Boolean values (0 or 1), its capabilities were minimal.

Linear Threshold Unit (LTU) Neuron (2nd ver.)

- Linear Threshold Unit (LTU) was introduced in 1957 which address the problem of McCulloch Pitts Neuron.
- In an LTU, weights are assigned to each event, and these weights can be negative or positive.
- The outcome of each even is still given a Boolean value (0 or 1), but then is multiplied with the assigned weight.



Linear Threshold Unit (LTU) Neuron (2nd ver.)

- **Problem:**
  - The weight are fixed
  - The output is still binary (0 or 1)
  - It can't be used to learn complex pattern available in the data.

Perceptron (3rd ver.)

- Perceptron is a binary classification algorithm for supervised learning and consists of a layer of LTUs.
- In a perceptron, LTUs use the same event outputs as input.
- The perceptron algorithm can adjust the weights to correct the behavior of the trained neural network.
- In addition, a bias term may be added to increase the accuracy performance of the network.
- It helps to classify the given input data; the output value is f(x) calculated as $f(x) = \langle w, x \rangle + b$
  - where w is a vector of weights and $\langle \cdot, \cdot \rangle$ denotes the dot product. We use the dot product as we are computing a weighted sum. The sign of $f(x)$ is used to classify x as either a positive or a negative instance.

Perceptron (3rd ver.)

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

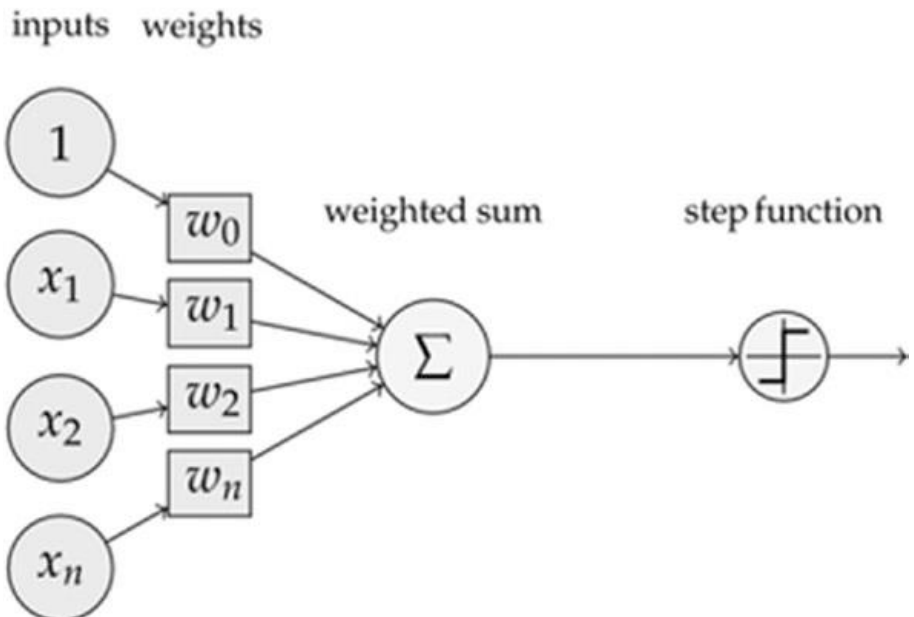
$$y = 1 \quad if \sum_{i=0}^{n} x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} x_i < 0$$

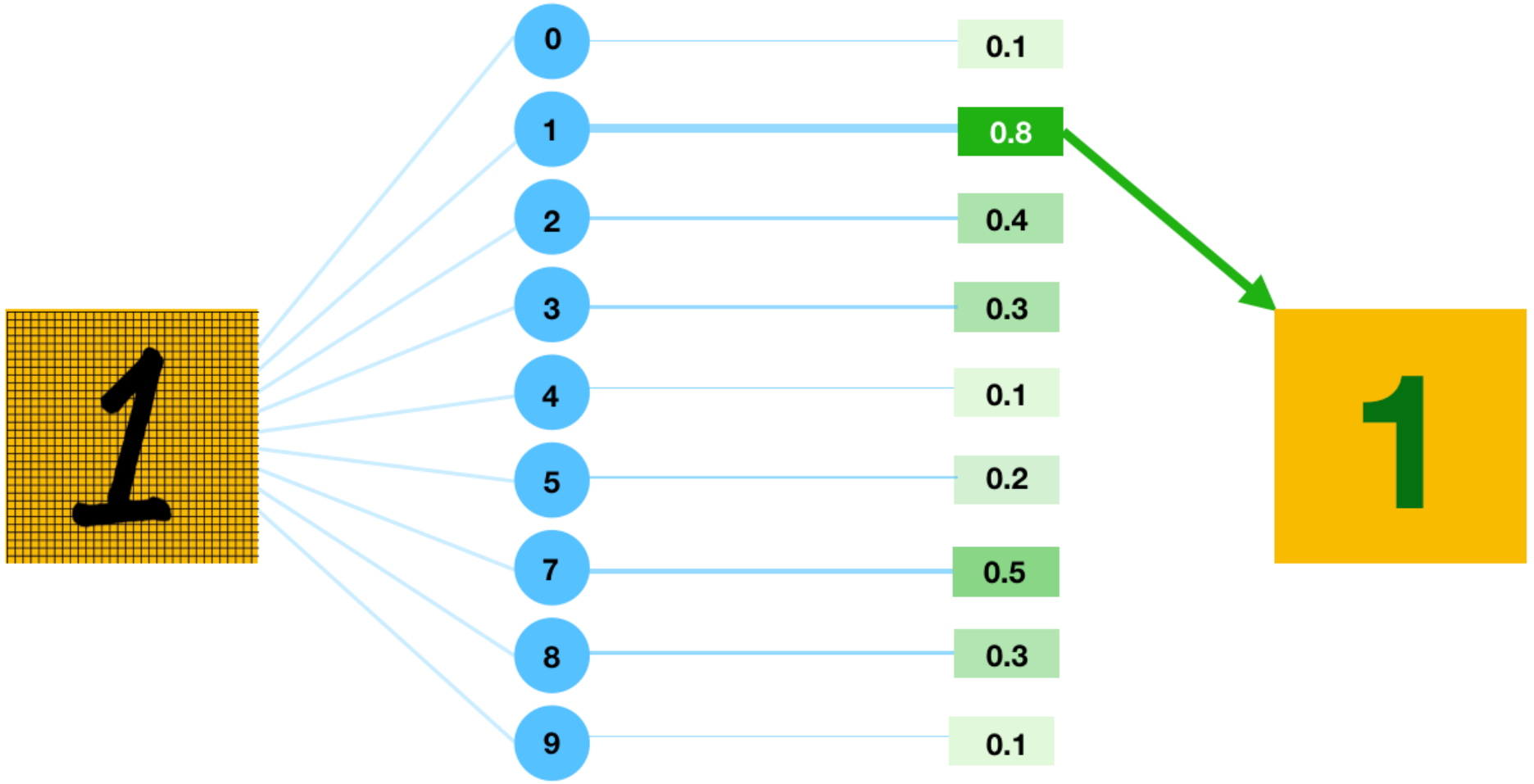
Perceptron

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

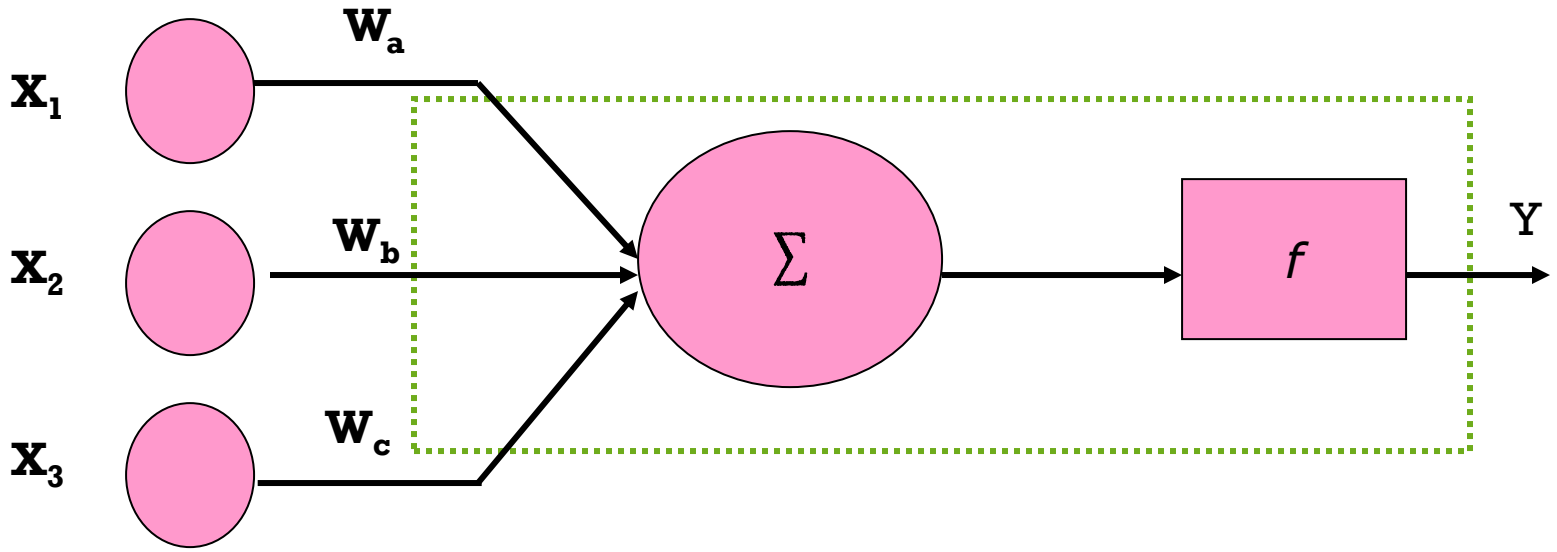
$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$



+ Perceptron (3rd ver.)



How ANN works?



Input units **Connection weights** **Summation function** **computation**

(dendrite) (synapse) (axon)

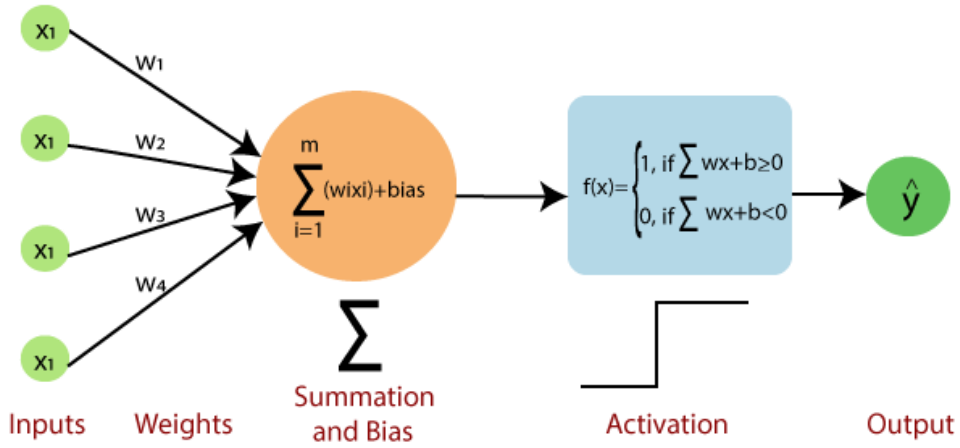
How ANN works?

- The artificial ***neuron receives one*** or ***more inputs*** and ***sums*** them to produce an ***output***.

- The ***sums*** of each ***node*** are ***weighted***, and the ***sum*** is ***passed*** through a ***non-linear function*** known as an ***activation function*** or ***transfer function***.

- The activation functions ***usually*** have a ***sigmoid shape***, but they may also take the form of other ***non-linear*** functions, ***piecewise linear functions***, or step ***functions.***

- "***Weighted sum***" of its input, ***adds a bias*** and then ***decides*** by ***activation*** function that whether it should be "***fired" or not***



$$Y = \sum (weight * input) + bias$$

Activity - Example

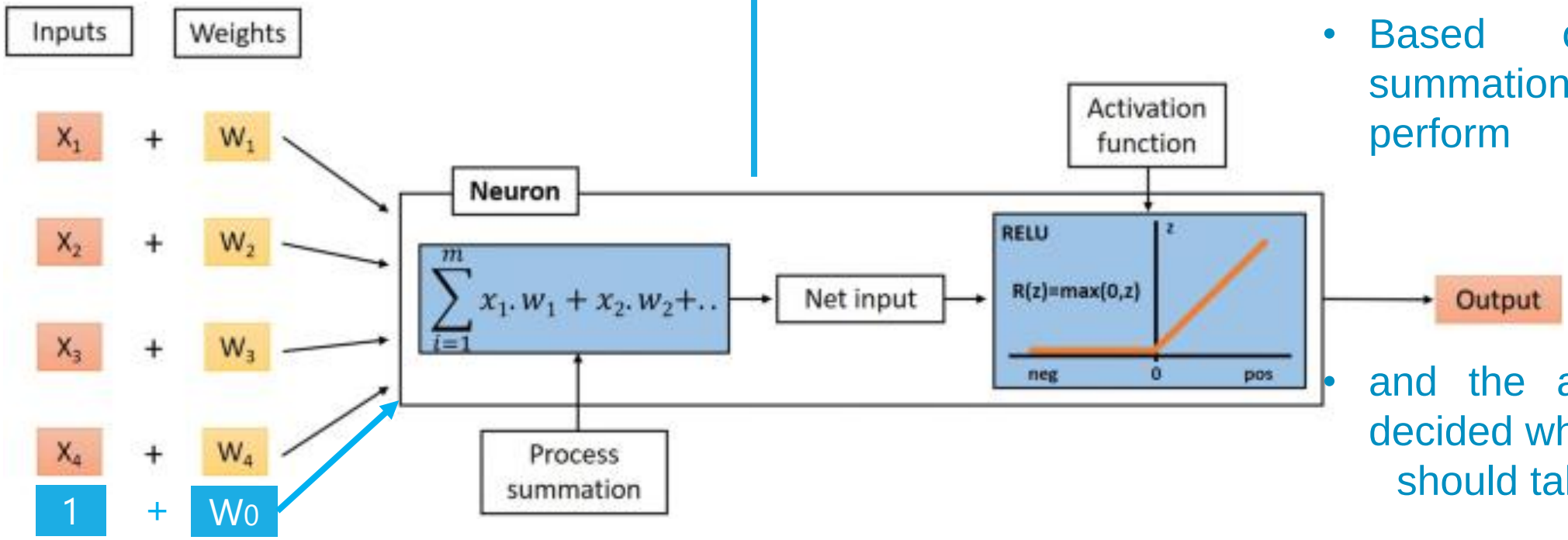
X1	X2	X3	X4	Y

Z = w1.x1 + w2.x2 + w3.x3 + x4.w4 + 1.w0

W = {w1, w2, w3, w4}

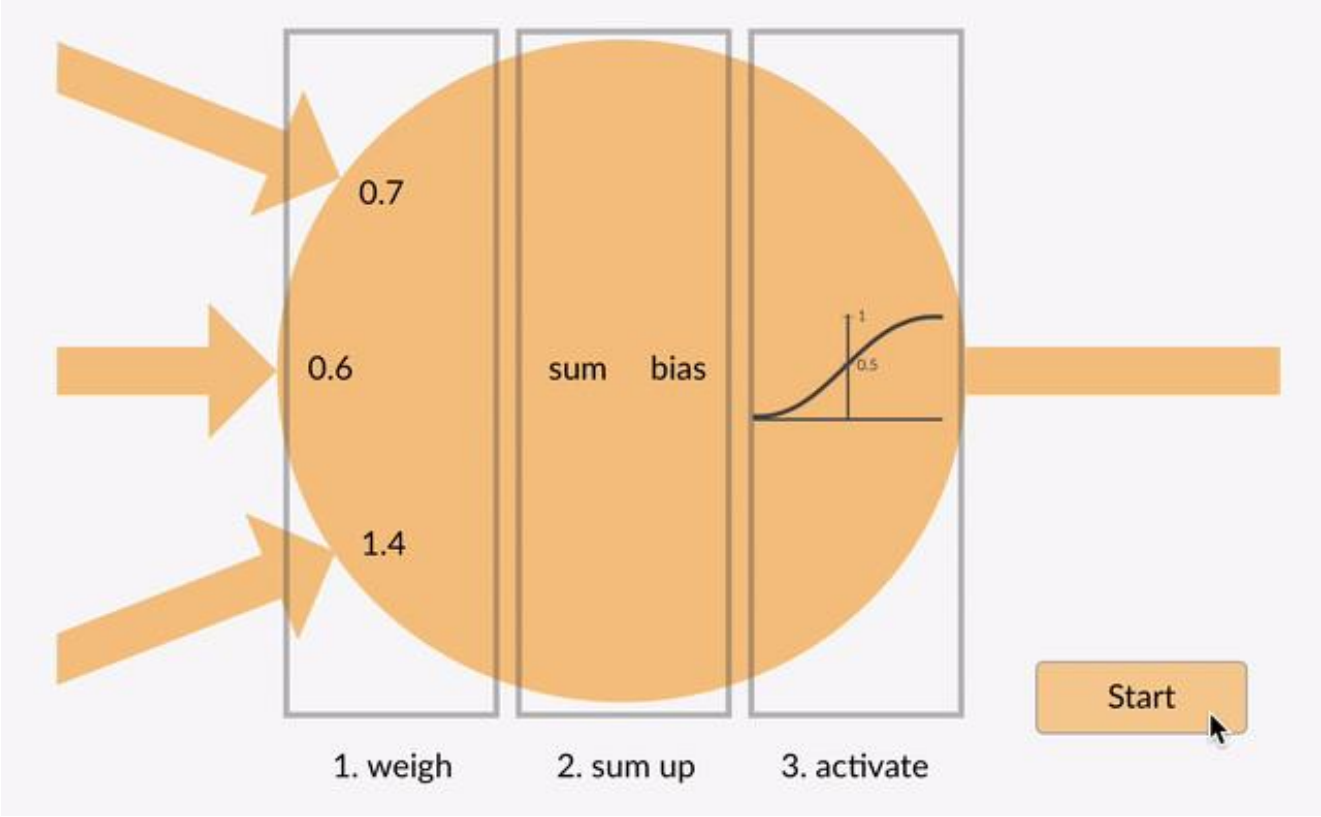
X = {x1, x2, x3, x4}

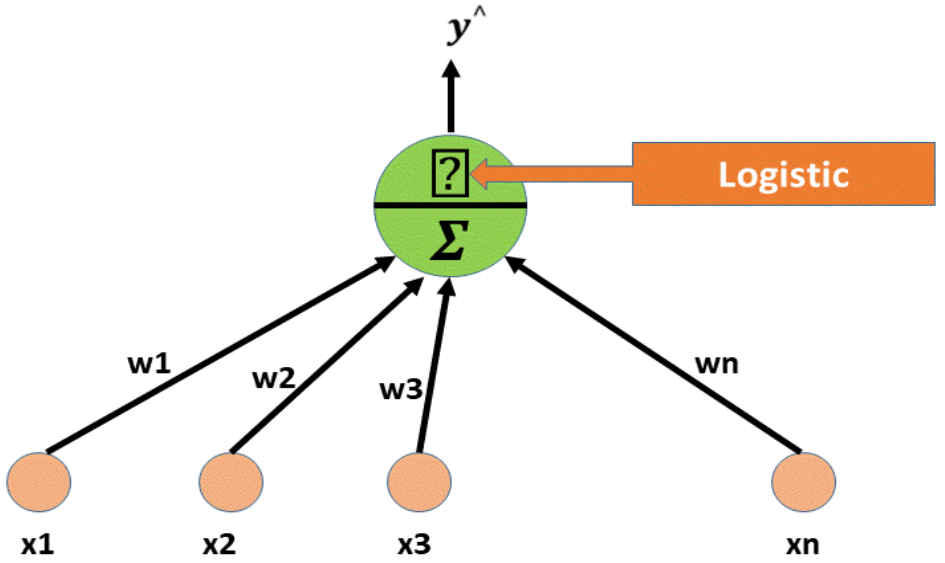
$$W^T.X + W_0$$



- Inputs are taking row by row
- W's are randomly assigned values
- Based on the input summation process are perform
- and the activation function decided whether the neuron should take action or not

Activation Functions

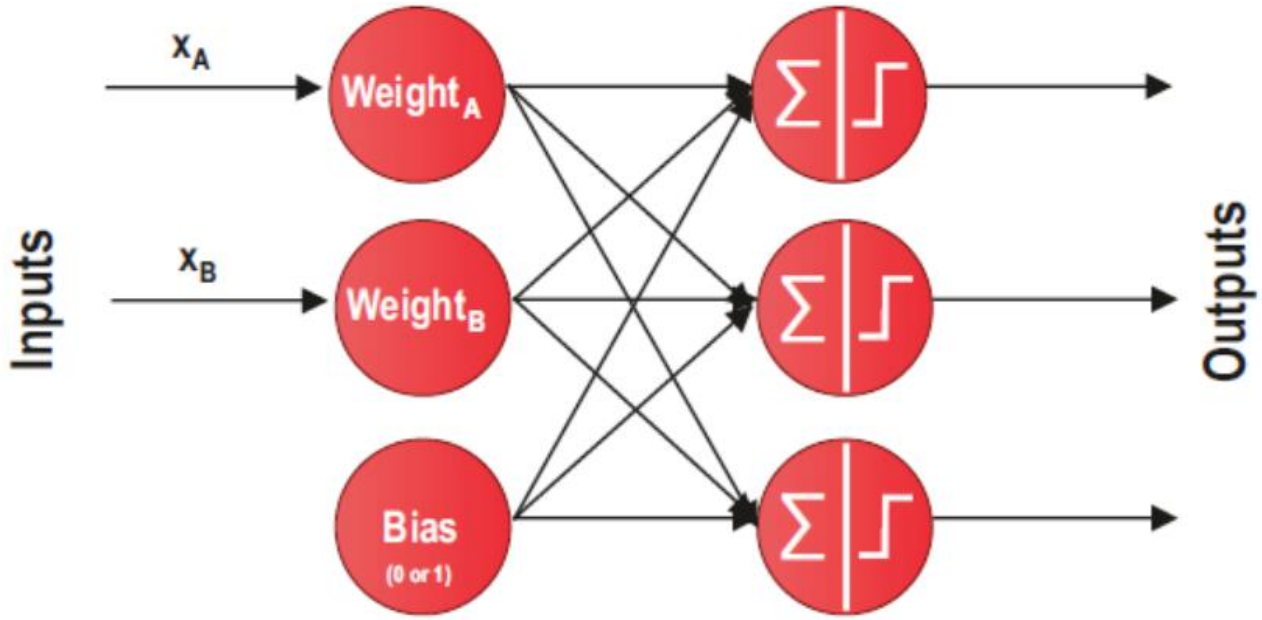




The placement of the activation function

Perceptron (3rd ver.)

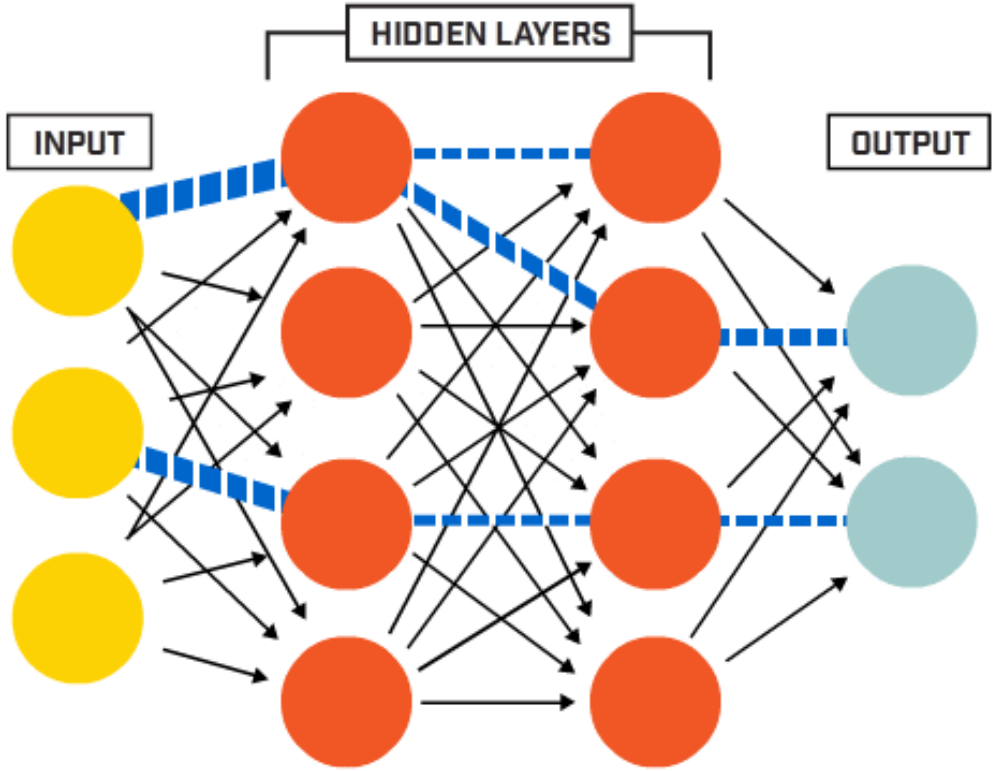
- **Single-Layer Perceptron:**
- Consist of one layer of perceptron
- There is one layer for outputs along with a single input layer that receives the inputs.



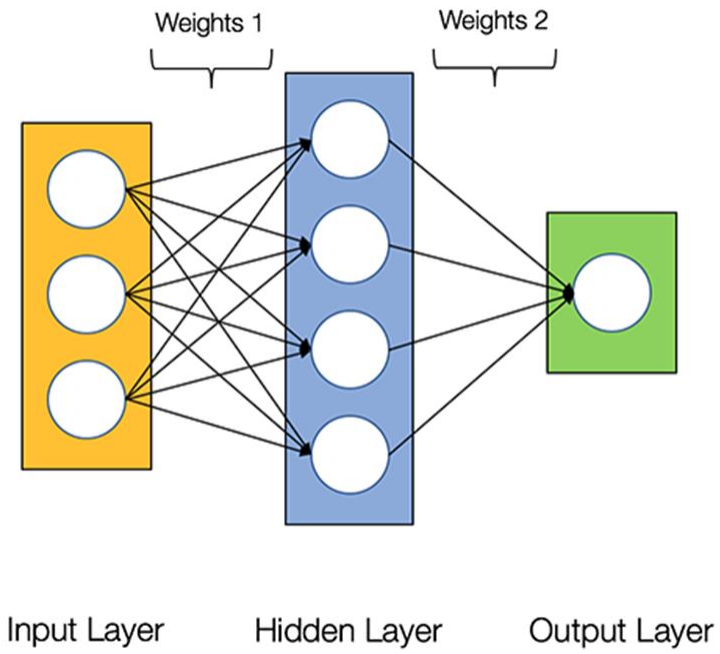
Perceptron (3rd ver.)

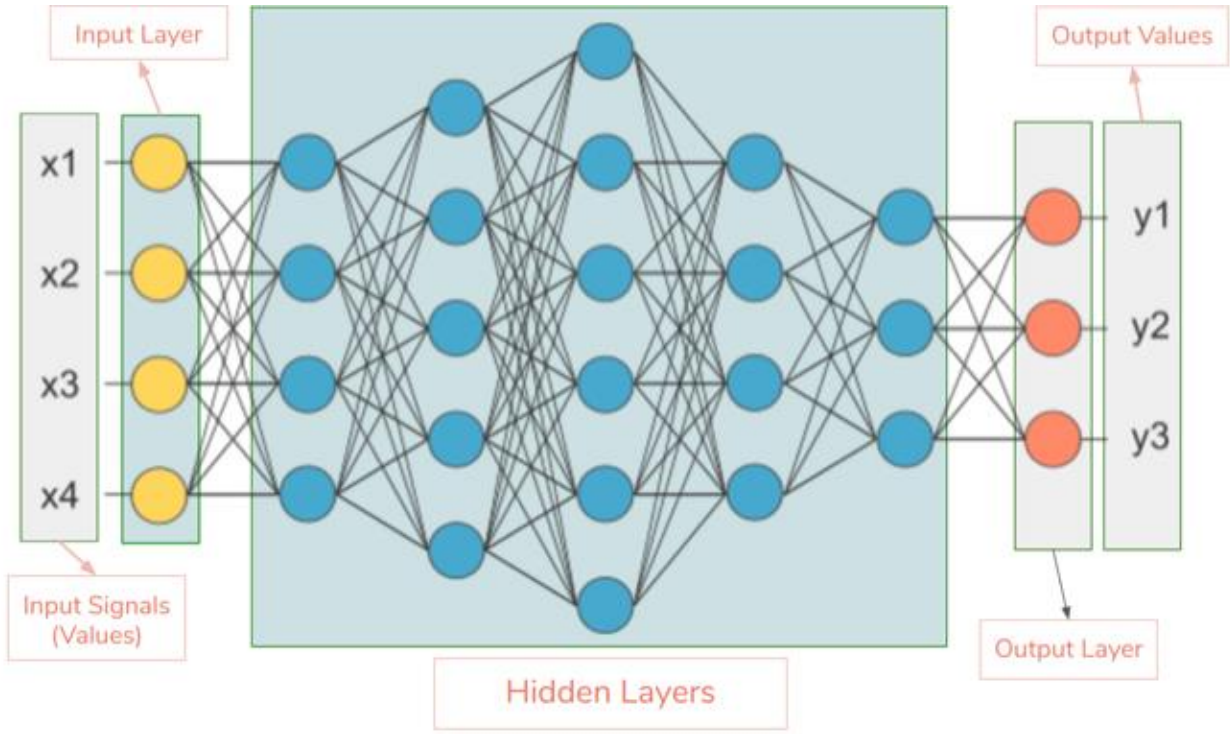
Multilayer Perceptron (MLP):

- When hidden layers are added to a single-layer perceptron.
- An MLP is considered as a type of deep neural network
- The artificial neural networks we build for everyday problems are examples of MLP



What is Neural Network?





Architecture of a Neural Network

Input layer is typically excluded when counting the number of layers in a Neural Network

What is Neural Network?

Input Layer:

- The nodes of the input layer are **passive**. (i.e., not modify the data),
- They receive value on their input and duplicate it to their multiple outputs.
- All of the input variables are represented as input nodes; each value from the input layer is duplicated and sent to all of the hidden nodes in first layer.

Hidden Layer(s):

- All of the input variables that came from the input layer are combined across one or more nodes (summation / activation) perform in the hidden layer.
- This creates new features from the input data and then pass these features to a new hidden layer and so on to get into the **output layer**

What is Neural Network?

Output Layer

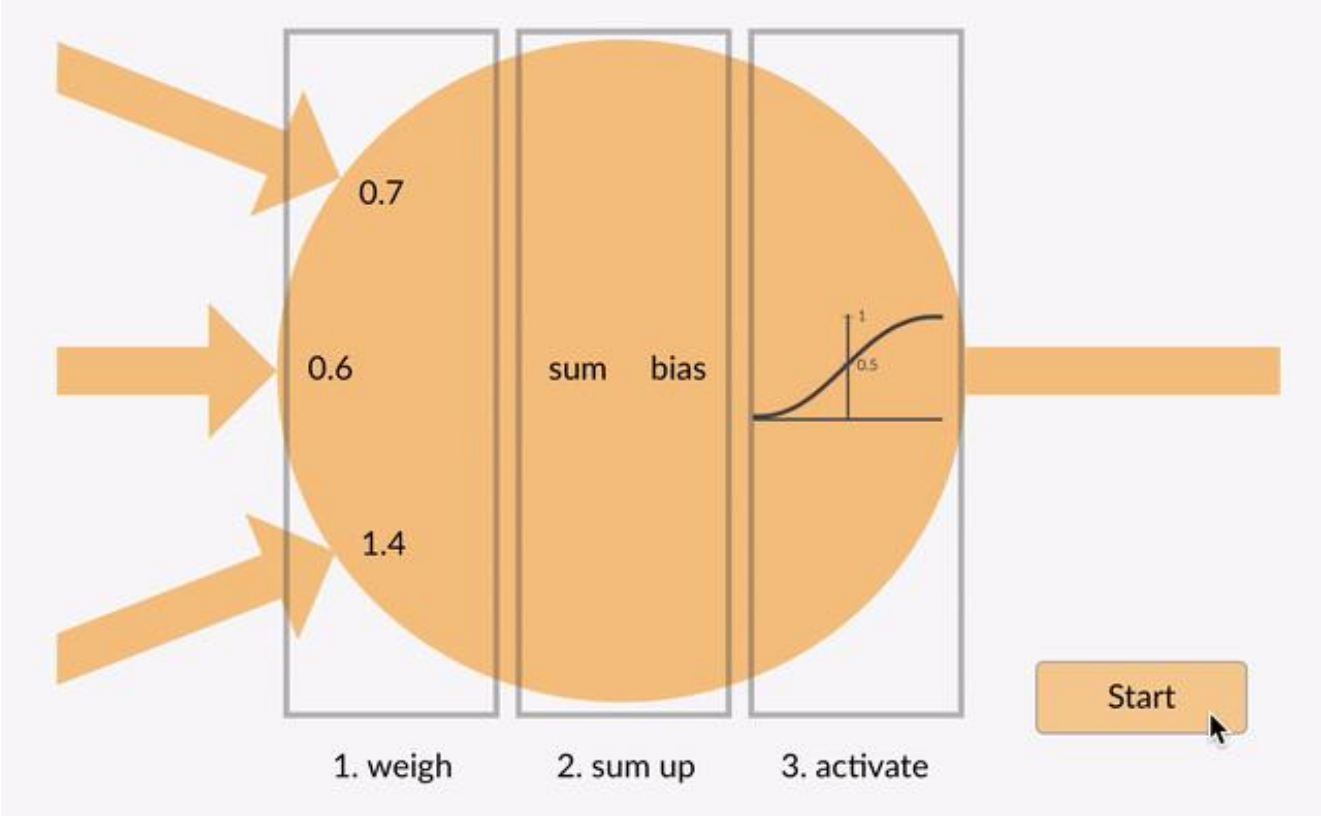
- Here we finally use an ***activation function*** that maps to the ***desired output format***

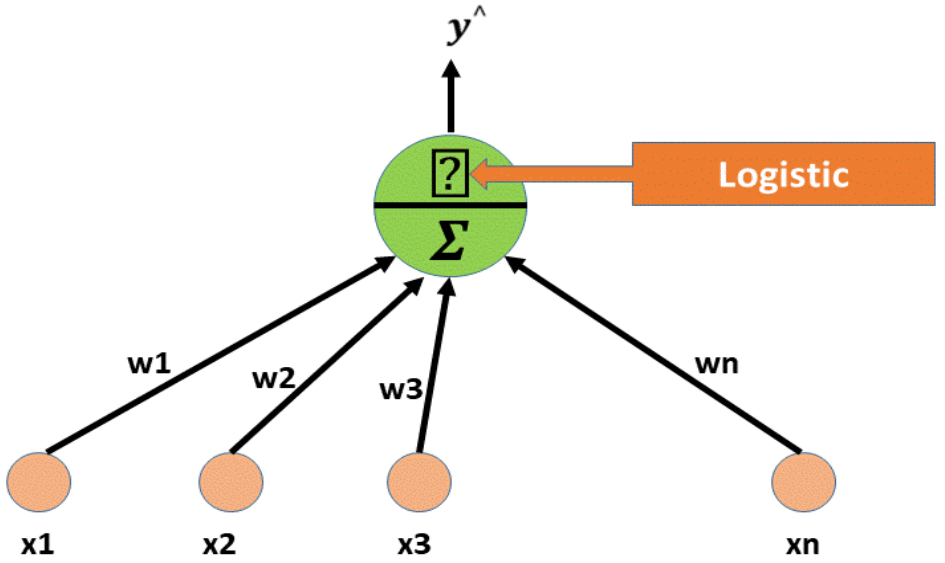
Hidden Layers

- Hidden neural network layers are set up in many different ways.
- In some cases, weighted inputs are randomly assigned.
- In other cases, they are fine-tuned and calibrated through a process ظبطها called **backpropagation**

Activation Functions

Activation Functions





The placement of the activation function

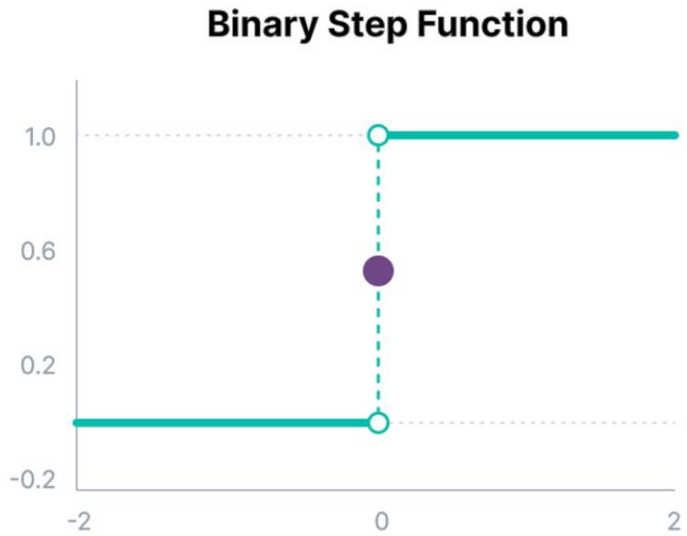
Activation function

Activation function ***decides***, ***whether*** a neuron should be ***activated*** or ***not*** by ***calculating weighted sum*** and further ***adding bias*** with it.

There are three types of activation functions:

- Binary step function
- Linear activation function
- Non-linear activation function

Activation function



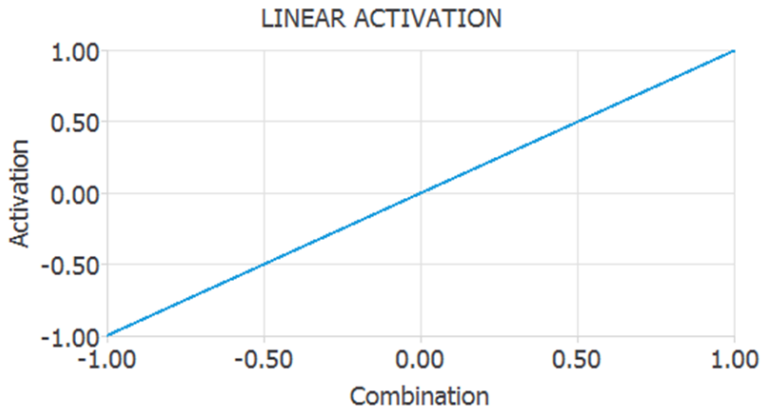
Binary Step Function:

- It is a threshold-based activation function.
- It check the output of neuron,
  - if the neuron output is above the threshold,
    - the activation neuron will send the same signal to the next layer
    - otherwise, will keep inactive the neuron. (or stop to participate…)

Problem with Binary Step Function:

- We can't use it in multiclassification problems. It's just say YES or NO
- It's only a trigger function and can't change any of the data that comes from the previous layer, so we need a more complex activation function.

Activation function



Linear Activation Function:

- It uses the function A = w ∗ x.
- It takes the inputs, multiplied by the weights for each neuron, and creates an output signal proportional to the input.
- In one sense, a linear function is better than a step function because it allows multiple outputs, not just **YES** and **NO**.

Problem with Linear Activation Function

- It's a constant function, so we can't use the backpropagation technique.
- It has limited power and ability to handle the complexity of varying parameters of input data

Activation function

Non-Linear Activation Function:

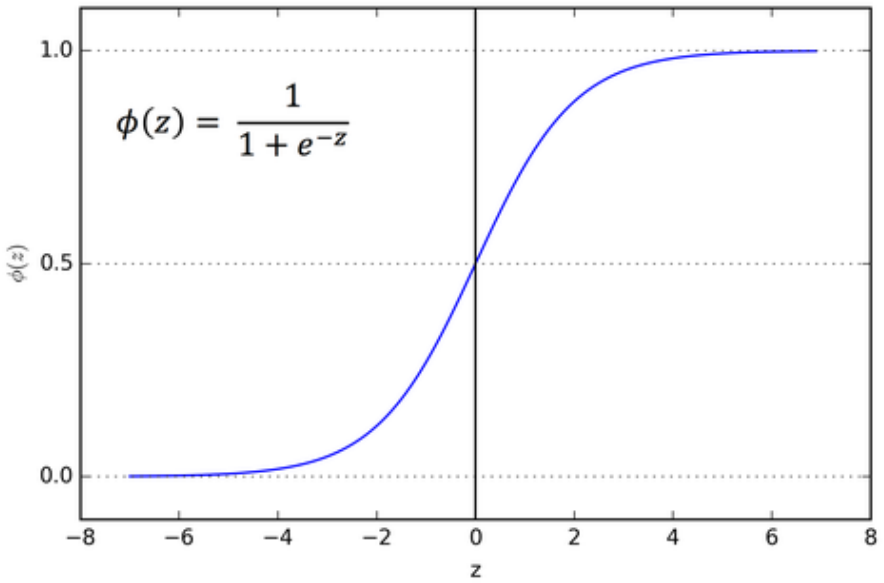
- Many neural network models use nonlinear activation functions.
- They allow the model to create complex mappings between the network's inputs and outputs, which are essential for learning and modeling complex data such as images, video, audio, and data sets that are nonlinear or have high dimensionality.
- The types of nonlinear activation functions that are used today in neural network models are
  - sigmoid/logistic
  - tanh/hyperbolic tangent
  - ReLU (rectified linear unit)
  - leaky ReLU
  - softmax
  - swish

Sigmoid Logistic Activation Function

It is a **function** which is plotted as **'S'** shaped graph.

- **Nature:** Non-linear.
- **Value Range:** 0 to 1
- **Uses:** Usually used in output layer of a ***binary classification***, where result is either 0 or 1, as value for sigmoid function lies between 0 and 1 only so, result can be predicted easily to be ***1*** if ***value is greater than 0.5 and 0 otherwise***.

$$Sigmoid(x) = \frac{1}{1+exp(-x)}$$

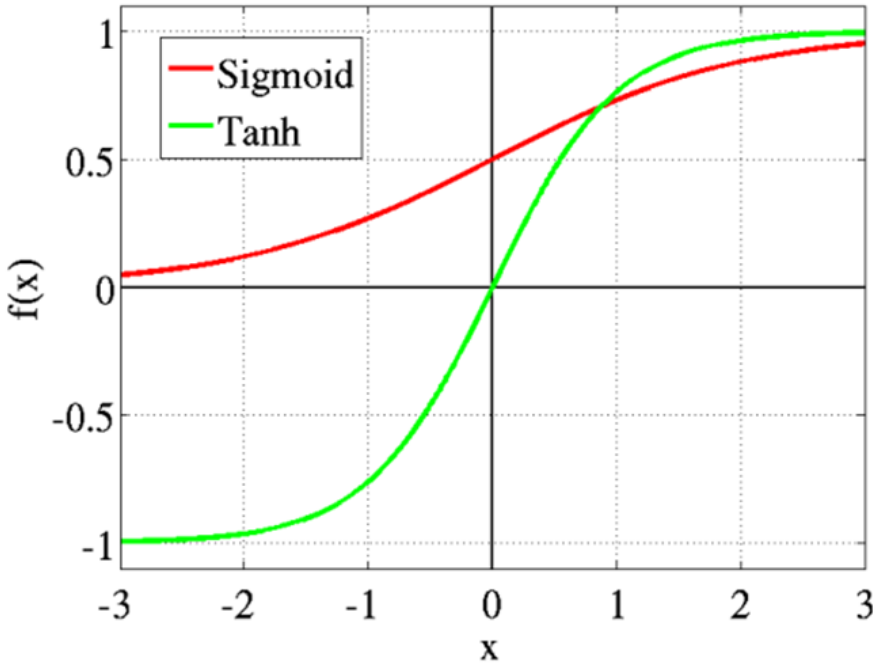


Tanh Function

The activation that works **almost always better** than **sigmoid function** is Tanh function also knows as **Tangent Hyperbolic function**.

$$tanh(x) = \frac{2}{2 + exp(-2x)} = 2 * sigmoid(2x) - 1$$

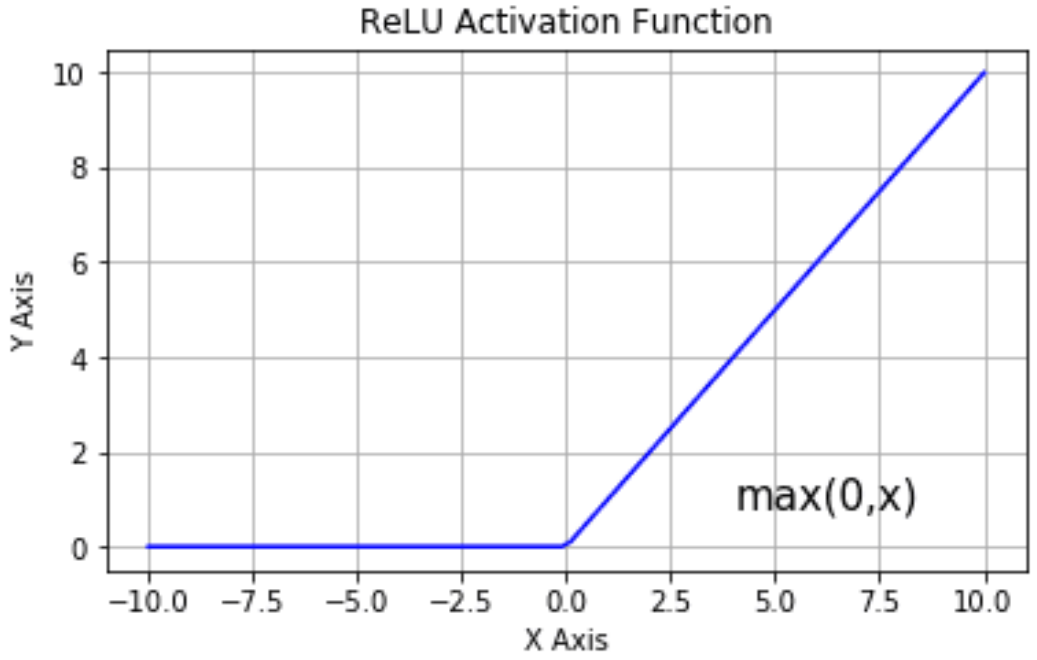
- **Value Range:** -1 to +1
- **Nature:** Non-linear
- **Uses:** Usually used in **hidden layers** of a neural network as its values lies between **-1 to 1**.
- The ***advantage*** is that the ***negative inputs*** will be ***mapped strongly negative,*** and the ***zero*** inputs will be ***mapped near zero*** in the tanh graph.
- ***Both tanh and logistic sigmoid activation functions are used in feed-forward nets.***



Rectified linear unit (ReLU)

- It is the ***most widely*** used ***activation function***. ***Implemented*** in ***hidden layers*** of Neural network.
- ***Value Range :-*** [0, inf)
- ***Nature :*** Non-Linear,
- ***Uses :- ReLU*** is ***less computationally expensive*** than ***tanh*** and ***sigmoid*** because it involves ***simpler mathematical*** operations.
- ***ReLU*** learns much ***faster*** than ***sigmoid*** and ***Tanh*** function.

$$Relu(x) = max(0, x)$$



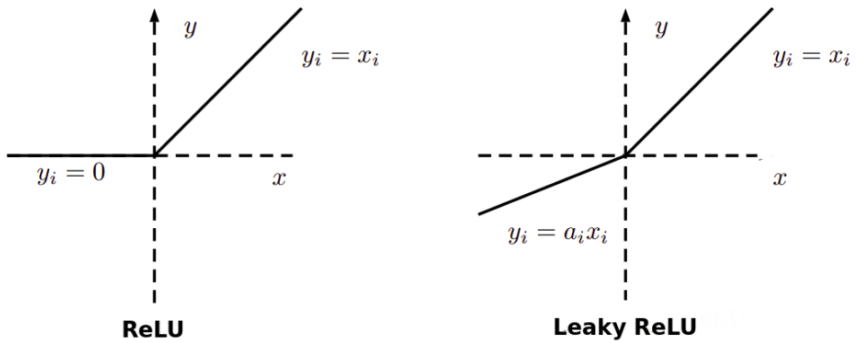
It is used in almost all the convolutional neural networks or deep learning.

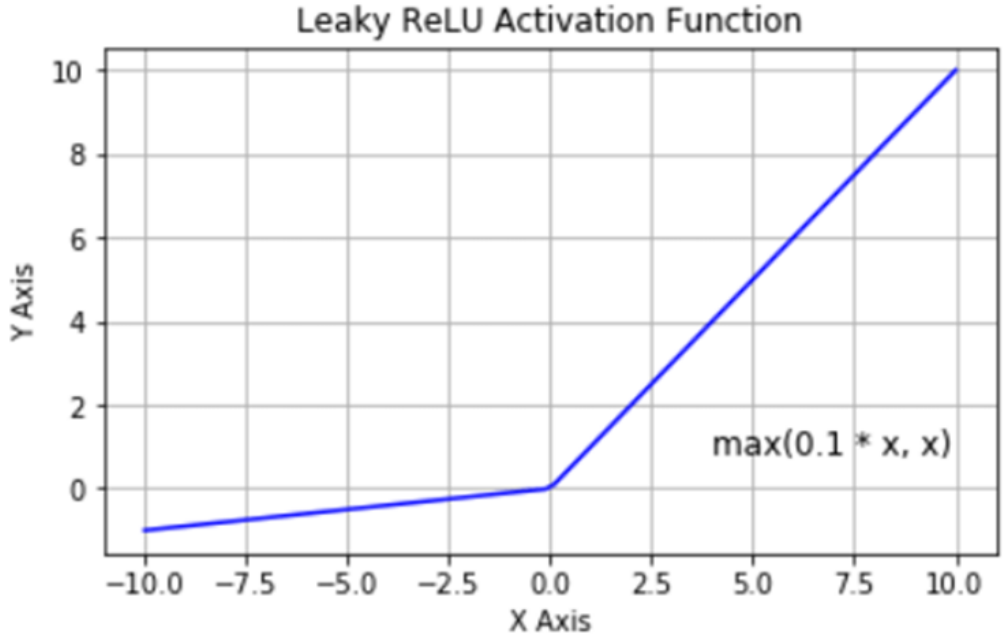
Leaky ReLU

- ReLU activation function is not taking care of the negative values and turn them into zero in the graph, which in turn making the neuron to not participate in the learning process of the neural network.
- Leaky ReLU is an **attempt** to solve the **dying ReLU** problem

$$LeakyRelu(x) = max(0.01 * x, x)$$

  - The **leak** helps to **increase the range** of the **ReLU** function.





Softmax Function

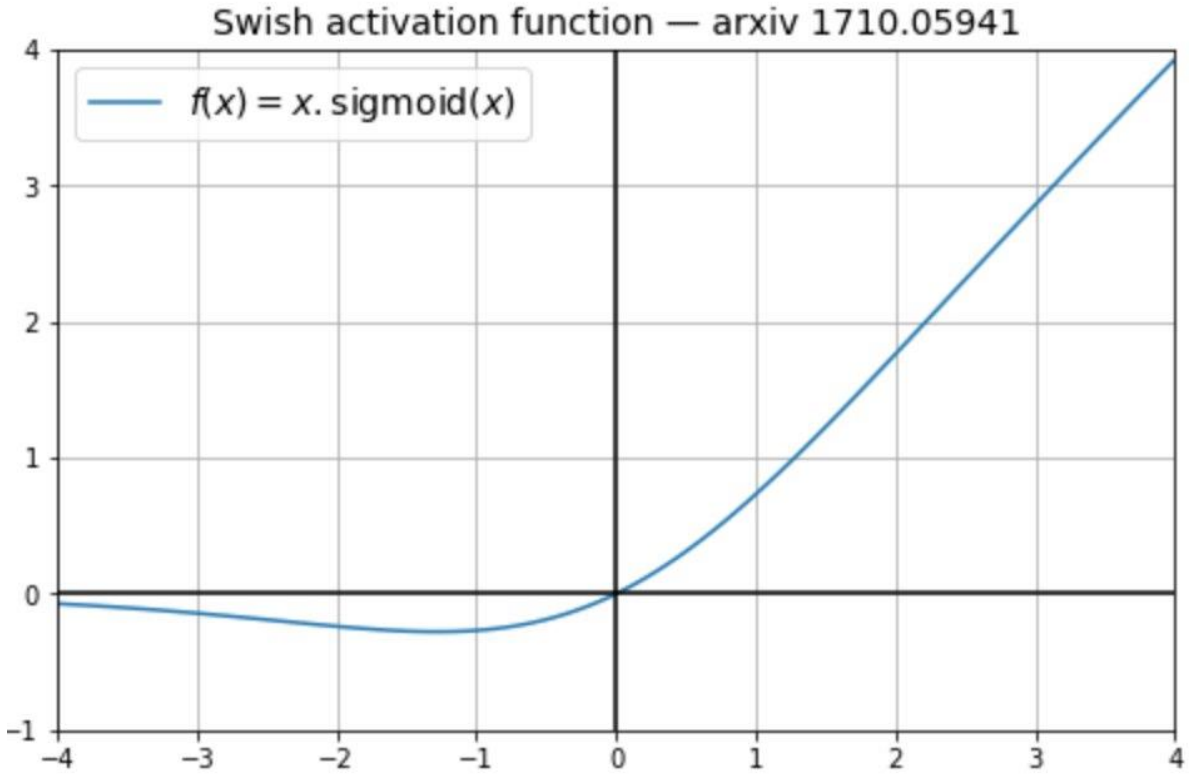
- The ***softmax function*** is same as ***sigmoid function***
- ***Nature:*** Non-Linear
- ***Uses:*** Usually used to ***handle multi-classification*** problems.
- ***Output:***
  - It calculate the probabilities of each target class and divided by all the possible target classes.
  - The class with high probability is the target class.

$$softmax(x) = \frac{exp(x)}{\sum_{j=0}^{k} exp(x_j)}$$

Swish Activation Function

- Google created this activation function called swish; it performs better than ReLU with a similar level of computational efficiency.

$$Swish(x) = \frac{x}{1 - exp(-x)}$$

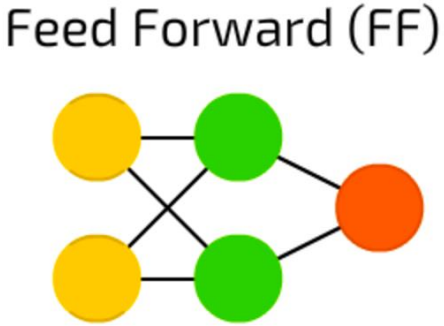


Choosing the Right Activation Function

- The basic rule of thumb is if you **really don't know** what activation function to use, then **simply use ReLU** as it is a **general activation** function and is used in most cases these days.
- We can use **Swish, Tanh** as well in replace of ReLU.
- If your output is for **binary classification** then, **sigmoid function** is very natural choice for output layer.
- The ***softmax function*** is a more ***generalized logistic*** activation function which is used for ***multiclass classification***.

Feedforward NN

Feedforward Neural Network

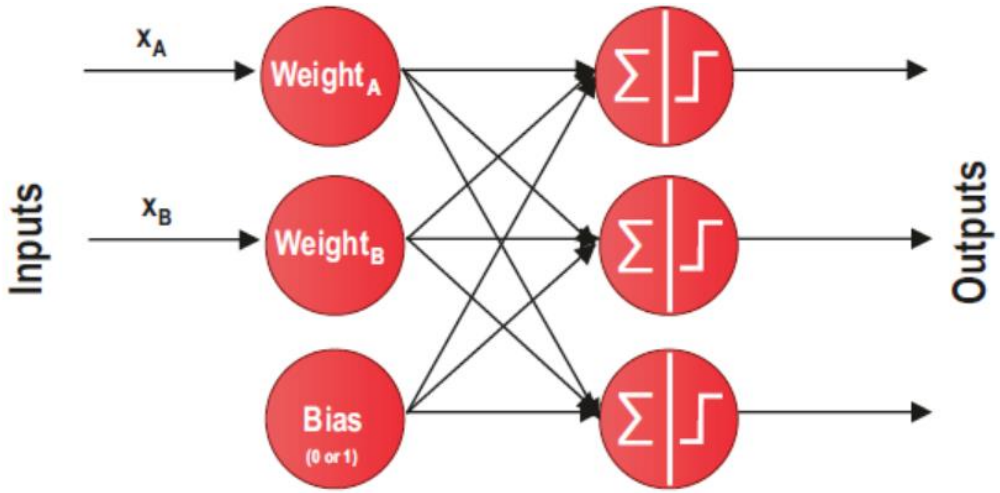


- All ***nodes*** are ***fully connected***
- Feed-forward NNs ***allow signals*** to travel ***one way*** only; from input to output.
- There is ***no feedback*** (loops) i.e., the ***output*** of any ***layer does not*** affect that ***same layer***.
- ***Activation flows*** from ***input layer*** to ***output***, ***without back loops***
- They are ***extensively used*** in ***pattern recognition***.
- In most cases this type of networks is ***trained*** using ***Backpropagation method***.

Feedforward Neural Network

Shallow Feedforward Neural Network (Single-layer Perceptron)

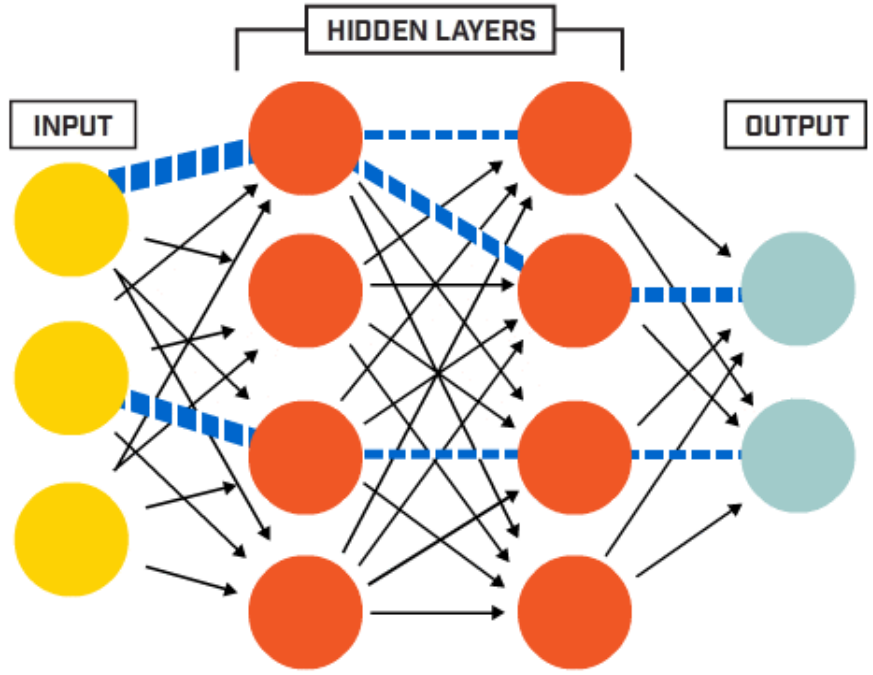
- This is the **simplest feedforward** neural Network and **does not** contain any **hidden** layer
- It only **consists** of a **single layer** of output nodes.
- We **do not** include the **input layer**, the reason for that is because at the **input layer no computations** is done, the inputs are fed directly to the outputs via a series of weights.



Feedforward Neural Network

Deep-Feedforward Neural Network (MLP - Multi-layer Perceptron)

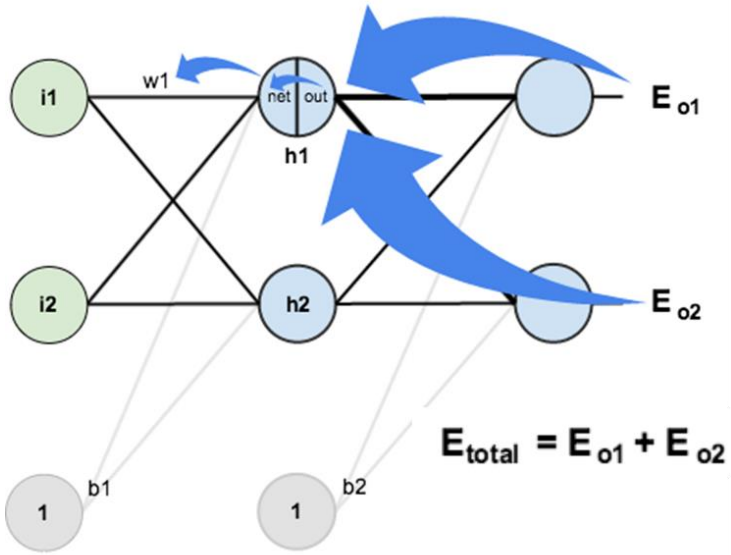
- This class of networks consists of ***multiple layers*** of computational units, usually ***interconnected*** in a ***feed-forward*** way.
- Each ***neuron*** in ***one layer*** has ***directed connections*** to the neurons of the ***subsequent layer***.
- In many applications the units of these networks apply a ***sigmoid function*** as an activation function.
- MLP are very more ***useful*** and one ***good*** reason is that, they are able to learn ***non-linear*** representations (most of the cases the data presented to us is not linearly separable).

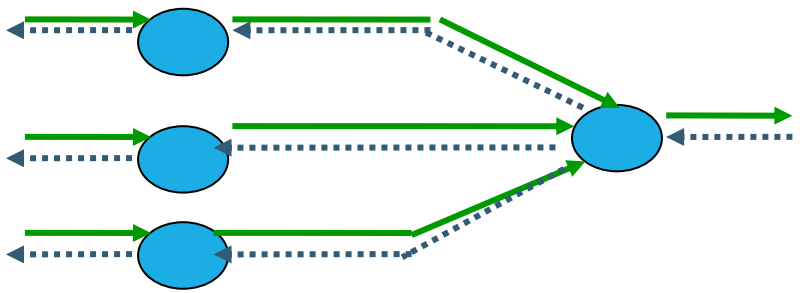


How are the weights initialized?

In general, ***initial weights*** are randomly chosen, with ***typical*** values between **-1.0 and 1.0 or -0.5 and 0.5**.

- There are two types of NNs. The first type is known as
  - **Fixed Networks (feedforward NN)**– where the weights are fixed
  - **Adaptive Networks (Backward Propagation NN)** – where the weights are changed to reduce prediction error.





Network activation Forward Step

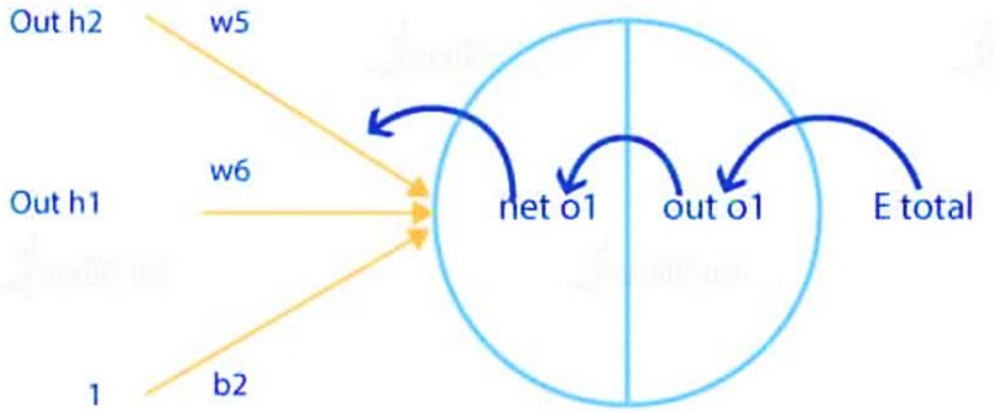
Error propagation Backward Step

Back Propagation

Backward propagation

- The goal with back propagation algorithm is to update each weight in the network so that the actual output is closer to the target output, thereby minimizing the error for each output neuron and the network as a whole.

- In Backward Propagation, we try to sequentially update the weights, first by making a forward pass on the network, after which we first update the weights of the last layer, using the label and last layer outputs, then subsequently use this information recursively on the layer just before and proceed.



Backward propagation

- Partial derivatives, chain rules, and linear algebra are the main tools required to deal with backpropagation
- For backpropagation to work, two basic assumptions are made regarding the error function:
  1. Total error can be written as a summation of individual errors of training samples/minibatch, $E = \sum^{k} E_k$
  2. Error can be written as a function of outputs of the network.

- Backpropagation consists of two parts:
  - Forward pass, wherein we initialize the weights and make a feedforward network to store all the values
  - Backward pass, which is performed to have the stored values and update the weights

Bw-Prop: Training a Single Perceptron

- Let's have dataset

x1	x2	x3	x4	y
1	2	5	6	1
3	4	5	5	8

- If we want to apply multiple regression on this dataset.

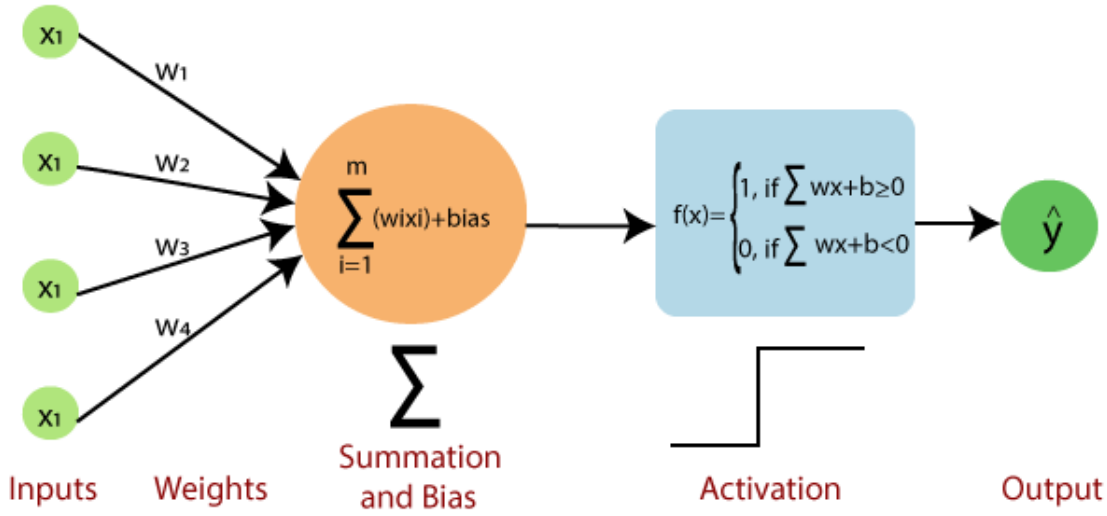
Q1. How many columns (features) , we have = _____

Bw-Prop: Training a Single Perceptron

- **What Linear Regression model will do?**
  - It will identify a hyperplane, which pass through these points/data (4D hyper-plan) and find the min-lost.
- **What loss function we use in LR model?**
  - The plan with min-lost

$$L = Loss = \frac{1}{N}\sum(y - \hat{y})^2$$

  - Mean-Square-Loss and we are applying this to the LR-model

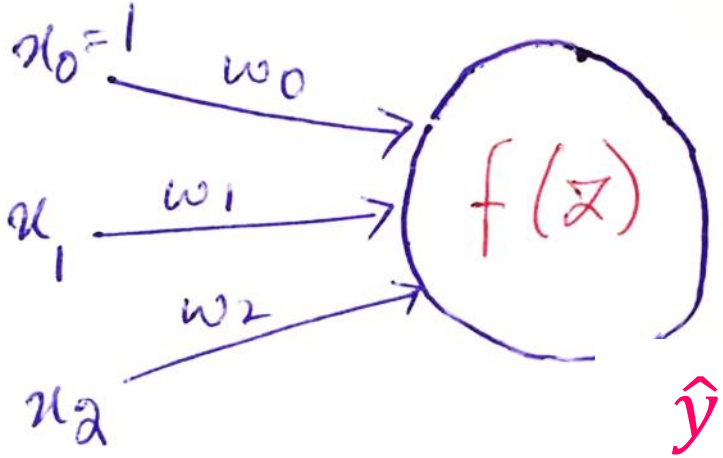


Single Perceptron Model

Bw-Prop: Training a Single Perceptron

- Single Perceptron Model

x_1	x_2	y
1	2	1
3	5	4
7	6	8



$L = loss = \frac{1}{2}(y - \hat{y})^2$

$\hat{y} = prediction$

$$z = w_1 x_1 + w_2 x_2 + w_0$$

$$\hat{y} = f(z)$$

Now we Calculate

$$L = (y - f(z))^2$$

Bw-Prop: Training a Single Perceptron

What we can do within the LR model using Back-Propagation?

$$z = w_1 x_1 + w_2 x_2 + w_0$$

$$\hat{y} = f(z)$$

$$L = (y - f(z))^2$$

- Answer: we need to identify best values for w1, w2.
- But how to identify them
- STEPS
  1. Initially we select some random values for weights w1, w2
  2. After substituting values, we find $\hat{y} = \boldsymbol{prediction}$, and LOSS
  3. Apply Stochastic Gradient Descent to update the weights

Bw-Prop: Training a Single Perceptron

Single Perceptron Model

- For the Backward Propagation, we need to updates the weights.
- First, we need to find the partial derivative of LOSS function by applying CHAIN-RULE.

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial w_1}$$

- Now calculate the new value for the weights using stochastic gradient descent algorithm

$$w_{1\,new} = w_{1\,old} - \frac{\partial L}{\partial w_1}$$

$$w_{2new} = w_{2\,old} - \frac{\partial L}{\partial w_2}.$$

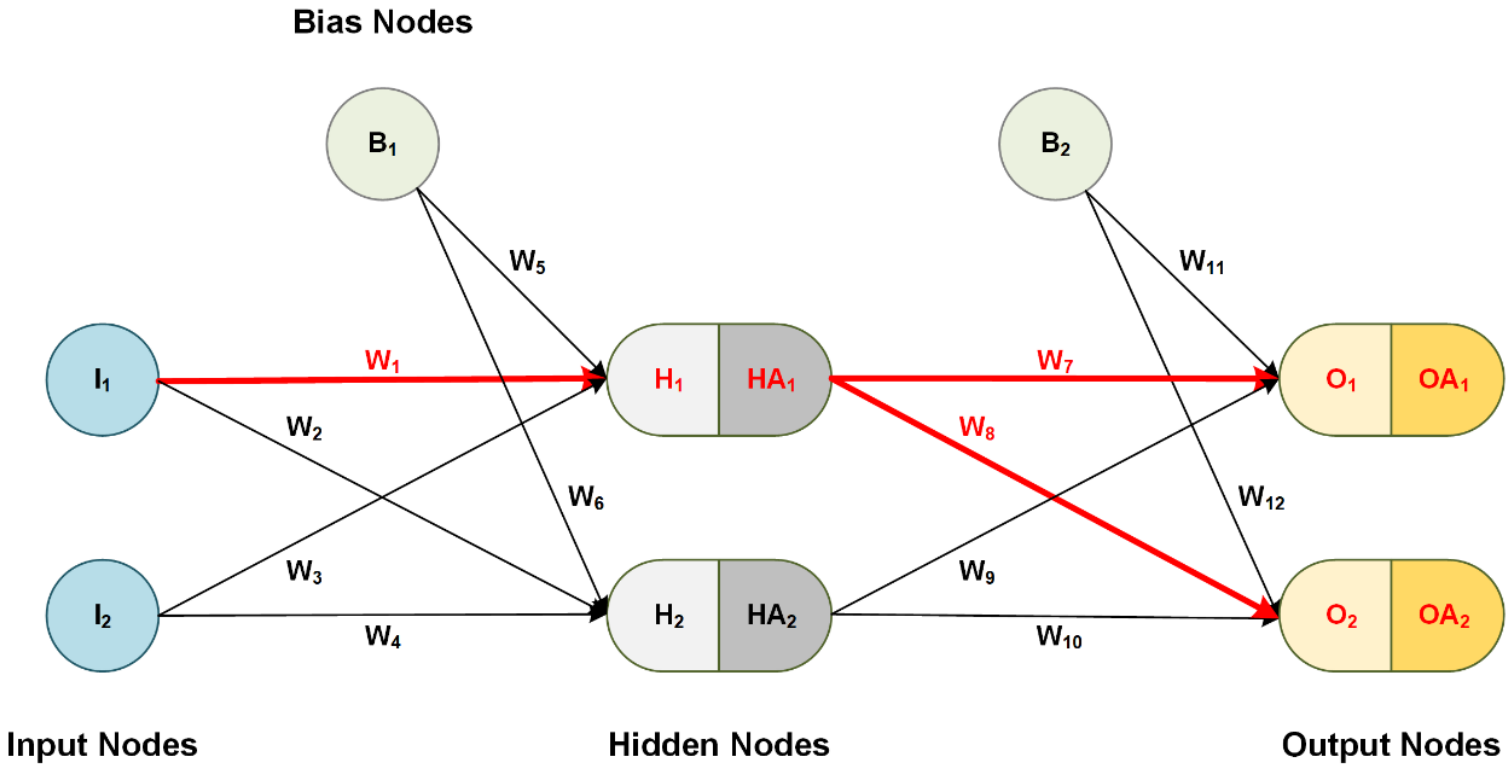
Bw-Prop: Training a Single Perceptron

- **How long we need to update these values using SGD?**
  - Ideally, updates W's till convergence (means no more updates in weights (w1, w2 and w0)
- **How to process data by Backward Propagation?**
  - The model take 1st set of values for x1, x2 and update the weights
  - Take the 2nd set of values for x1, x2 and update the weights.
  - ....
  - This process continue until all records are processed and we called it 1st epoch… followed by same process for 2nd epoch and so on.

Bw-Prop: Training a Single Perceptron

- **What is Backward Propagation?**
  - We are taken input (record by record)
  - pass through the model,
  - perform activation function,
  - calculate the $\hat{y}$, and then LOSS function and
  - using SGD to update the weights.
  - This is called Backward Propagation.

Bw-Prop: Deep Model



$$\frac{\partial e}{\partial W_1} = \underbrace{\left(\frac{\partial e}{\partial OA_1} \frac{\partial OA_1}{\partial O_1} \frac{\partial O_1}{\partial HA_1} \frac{\partial HA_1}{\partial H_1} \frac{\partial H_1}{\partial W_1} \right)}_{(1)} + \underbrace{\left(\frac{\partial e}{\partial OA_2} \frac{\partial OA_2}{\partial O_2} \frac{\partial O_2}{\partial HA_1} \frac{\partial HA_1}{\partial H_1} \frac{\partial H_1}{\partial W_1} \right)}_{(2)}$$

Role of Activation Function in Back Propagation

Role of Activation Function in Back-Propagation

- **Question:** How we can update the weights of the deep neural net model?
- **Answer:** Stochastic Gradient Decent algorithm, actually play a role to calculate value and update the weights of the model by applying partial derivative process.

Step-Function (Binary Function): Problems?

- As we know that the output of this activation function is either one or zero which are constant values,
- So, during the Back-Propagation process, if you take the derivative of the constant values, the answer is 0.
- It cannot help us to update the weights of the model (with backward propagation).

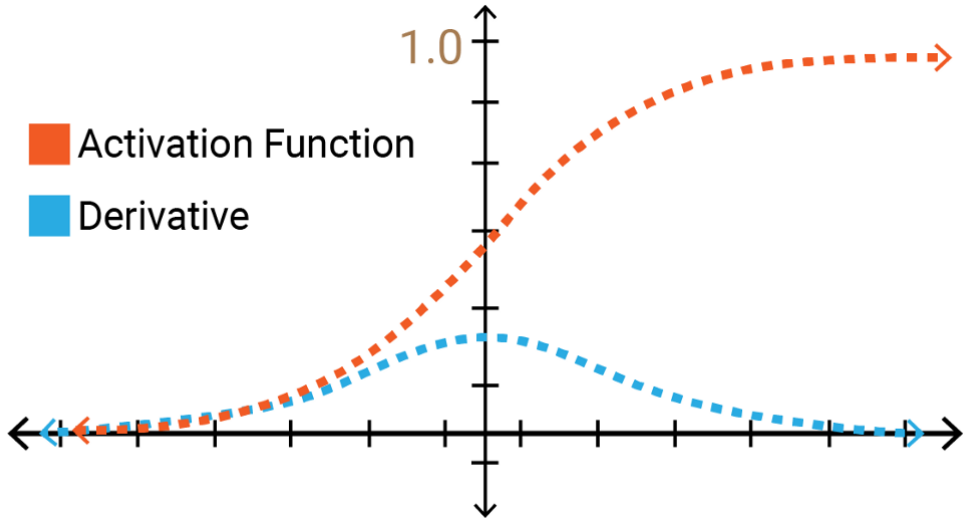
Role of Activation Function in Back-Propagation

Linear Function: Problems?

- As we know that the output of this activation function is $A = w * x$,
- So, during the Back-Propagation process, if you take the **derivative** a constant values is generated.
- It can update the weights of the model (with backward propagation) with the previous values.
- Hence no-learning is performed with this activation function.

Role of Activation Function in Back-Propagation

- **Sigmoid Activation Function: Problem**
- **Problem-1: Vanishing Gradients**
  - With this activation function, the gradient values are significant for range -3 and 3 but the graph gets much flatter in other regions.
  - This implies that for values greater than 3 or less than -3, will have very small gradients. As the gradient value approaches zero, the network is not really learning.



Issues: (1) less contributing in learning as it is very slow in the weights updates (2) computationally very costly and time consuming.

Role of Activation Function in Back-Propagation

- **Sigmoid Activation Function: Problem**
- **Problem-2: Sigmoid is not symmetric around Zero**
  - We're providing any value to sigmoid functions, but we are getting the answer between zero and one, always getting positive answer, which make it not symmetric around zero.
  - The output of all the neurons will be of the same sign.

Role of Activation Function in Back-Propagation

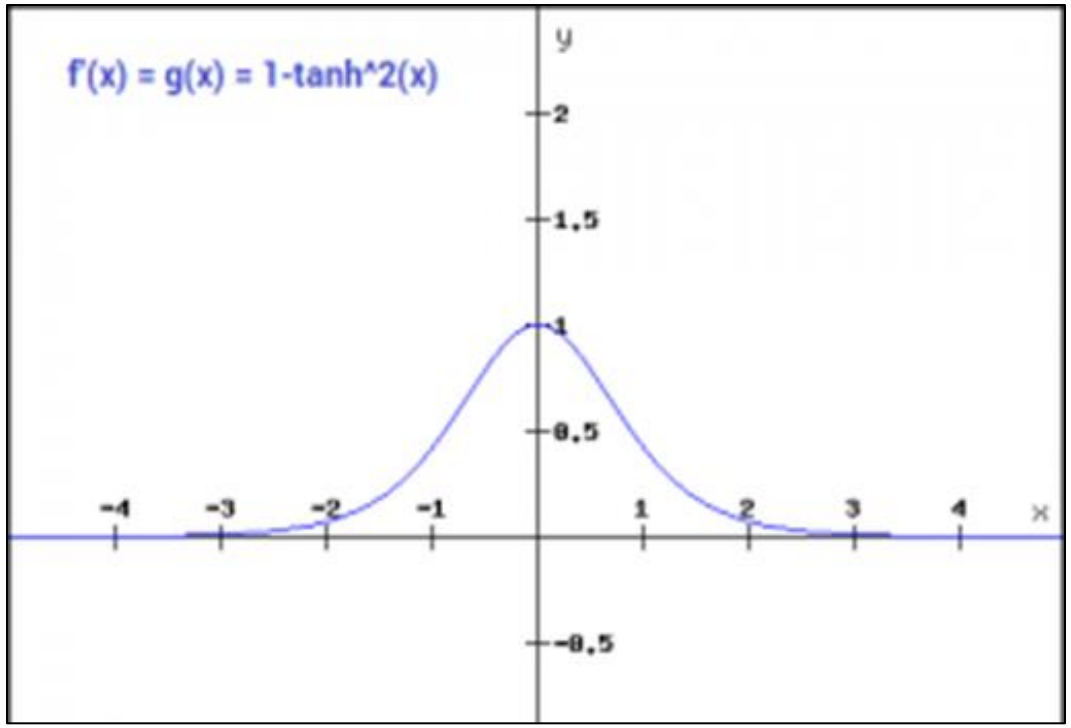
- **Tanh Activation Function: Problem**

- It solve symmetric around 0 issue, but face **Problem-1: Vanishing Gradients**

Issues:

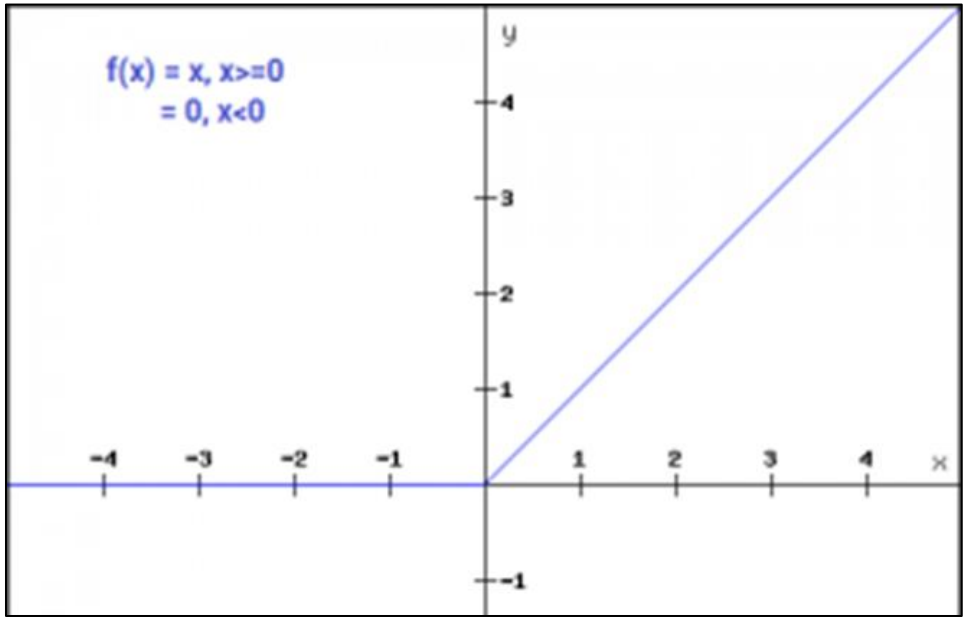
(1) less contributing in learning as it is very slow in the weights updates

(2) computationally very costly and time consuming.



Role of Activation Function in Back-Propagation

- **ReLU Activation Function: Problem**
- It is far more computationally efficient when compared to the sigmoid and tanh function.
- **Problem-1: Dead Neurons**
- As it's not dealing negative values, the gradient value is zero.
- Due to this reason, during the Back-Propogation, the weights and biases for some neurons are not updated. This can create dead neurons which never get activated.



$$f(x)=\max(0,x)$$

Role of Activation Function in Back-Propagation

- **Leaky ReLU Activation Function:**

```
f(x)= 0.01x, x<0
    =    x, x>=0
```

- Multiply a small value with input incase of negative values to avoid dead neurons situation.

- **Swish Activation Function:**
  - Swish is as computationally efficient as ReLU and shows better performance than ReLU on deeper models.
  - The values for swish ranges from negative infinity to infinity.

- **Softmax Activation Function:**
  - Softmax function is often described as a combination of multiple sigmoids and is used for multi-class problems.

Choosing the right Activation Function

- Depending upon the properties of the problem we might be able to make a better choice for easy and quicker convergence of the network.
  - Sigmoid functions and their combinations generally work better in the case of binary classifiers
  - Sigmoid and tanh functions are sometimes avoided due to the vanishing gradient problem
  - ReLU function is a general activation function and is used in most cases these days. If we encounter a case of dead neurons, then leaky ReLU function is the best choice
  - Always keep in mind that ReLU function should only be used in the hidden layers
  - Swish is also recommended in place of ReLU
  - Softmax is used in output layer for multi-classification problem.

Deep-NN Model

Steps to Create Deep Neural Network

Four basic things is required to build a Deep Neural Network Model

1. Model Creation (using Keras API [Sequential, Functional])
2. Model Compilation (model.compile(parameters))
3. Model Training (model.fit(parameters))
4. Model Evaluate (model.evaluate(X_test, Y_test))
5. Model Prediction (model.predict(X_test))

For more details, visit these web or Search example code over github

https://keras.io/examples/structured_data/structured_data_classification_from_scratch/

https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/

Applications of ANN

- Signal processing
- Pattern recognition, e.g. handwritten characters or face identification.
- Speech recognition
- Human Emotion Detection
- Sales forecasting
- Industrial process control
- Risk management
- Medical diagnosis
- Texture analysis

Pros of ANN

- It can model **non-linear systems**
- The ability to learn allows the network to adapt to changes in the surrounding environment
- It can provide a **confidence level to a given solution**
- Can ***learn*** more ***complicated class boundaries***
- Can handle ***large number*** of features
- Have ***noise tolerance***

Cons of ANN

- ***Long training Time***
- ***Hard*** to ***interpret***
- ***Hard*** to ***implement***: ***trail*** and ***error*** for ***choosing number*** of nodes, adjusting weights

End of Lecture - 07